OTA_SYSTEMS_RESEARCH

Comprehensive OTA (Over-The-Air) Update Systems Research

> **Date:** March 2026
> **Purpose:** Foundation for designing a custom OTA system targeting Android 15 on RK3588 (Orange Pi 5 Max)
> **Scope:** Exhaustive survey of all major open-source and commercial OTA solutions

---

Table of Contents

---

1. Android OTA Systems

1.1 AOSP Update Engine (update_engine)

Architecture Overview: - The `update_engine` daemon (`system/update_engine/` in AOSP) is the core component responsible for applying A/B (seamless) updates on Android and ChromeOS. - Runs as a background daemon with low I/O priority to avoid disrupting the user experience. - Shared codebase between Android and ChromeOS.

Key Components & Data Flow: 1. **Update Check:** The `UpdateCheckScheduler` periodically polls the update server (Omaha protocol) for available updates. 2. **Download:** The `DownloadAction` streams the update payload directly from network to the inactive partition — no intermediate cache storage is needed. 3. **Install:** The `InstallPlan` drives `FilesystemVerifierAction` and `ApplyPayloadAction` which apply operations (REPLACE, REPLACE_BZ,

REPLACE_XZ, ZERO, SOURCE_COPY, SOURCE_BSDIFF, PUFFDIFF) to the inactive slot. 4. **Post-install:** After writing, partitions are mounted at `/postinstall` for verification. 5. **Boot Control:** Sets the active slot via `IBootControl` HAL, marking the new partition as bootable and setting the old one as uninstallable.

Payload Format: - An OTA zip contains `payload.bin`, `payload_properties.txt`, and `care_map.pb`. - `payload.bin` structure: Header (magic, version, manifest length, manifest signature, metadata signature) → Manifest (list of partition operations) → Extra data blobs. - The manifest describes operations per partition with source/target digest verification (SHA-256).

Update Verification Mechanisms: - SHA-256 hash verification of source and target partitions. - RSA signature verification of the payload manifest (Google's key or OEM key). - Post-install verification: partitions are mounted and checked for basic bootability. - dm-verity verification on boot. - `care_map.pb` defines which blocks need verity check after update.

Rollback Capabilities: - A/B design inherently supports rollback: if the new slot fails to boot, the bootloader automatically falls back to the old slot. - Boot control HAL manages `unbootable` flag — if new slot fails to boot N times, it's marked unbootable. - `snippet` rollback protection via Android Verified Boot (AVB).

Rollout/Phased Deployment: - Omaha protocol supports fractional rollout — the server returns "no update" to a percentage of devices. - Google Play Services manages rollout percentages server-side. - Staged rollout: 1% → 5% → 10% → 25% → 50% → 100%.

Client-Server Protocol: - **Omaha Protocol** (Google's update protocol): XML-based request/response over HTTPS. - Request includes: app_id, version, hardware_class, language, delta_ok, additional attributes. - Response includes: status, update check result, URLs, manifest, hash.

Language/Technology Stack: - C++ (core daemon), Brillo (ChromeOS derivative), DBus IPC, Binder (Android). - Build: Soong/Blueprint.

License: Apache 2.0 (AOSP)

Scalability: Google manages billions of Android devices; server infrastructure scales horizontally. Client is lightweight.

Strengths: - Battle-tested at Google scale (billions of devices). - Seamless, no-downtime updates. - Built-in rollback protection. - Delta updates supported (reduces bandwidth). - Integrated with Android security model (dm-verity, AVB).

Weaknesses: - Complex codebase; difficult to modify or extend. - Tightly coupled to AOSP build system. - Omaha protocol is XML-heavy and somewhat opaque. - Requires dual partition slots (A/B) or Virtual A/B. - Server-side infrastructure is not open-sourced.

Android 15 / RK3588 Compatibility: Full support — this IS the Android OTA mechanism. Must be used or wrapped.

Encapsulatable: Partially. The update_engine client can be driven by a custom server (replace Omaha), but the payload format and partition layout must be maintained.

1.2 Android Recovery Mode OTA

Architecture Overview: - The legacy (non-A/B) update mechanism, now deprecated starting Android 15. - Device reboots into a special recovery mode to apply the update. - Update package is downloaded to `/cache` or `/data` partition while the system is running.

Key Components & Data Flow: 1. **Download:** System UI downloads OTA zip to `/cache` or `/data/ota/`. 2. **Reboot to Recovery:** `RecoverySystem.installPackage()` triggers reboot to recovery. 3. **Verification:** Recovery verifies the OTA package signature (RSA against embedded certificates in `/system/etc/security/otacerts.zip`). 4. **Apply:** `edify` scripting language (`updater` binary) executes the update script inside the zip. 5. **Reboot:** Device reboots into the updated system.

Update Verification: - Package signature verification (RSA). - `assert()` commands in the updater script verify device compatibility. - Post-update dm-verity handles runtime integrity.

Rollback Capabilities: - Limited. If update fails mid-application, device may be bricked. - No automatic rollback — manual reflash required. - Some implementations use a `backup` partition to store the old system image.

Rollout/Phased Deployment: - Same Omaha protocol on server side. - Client just downloads and applies when told.

Client-Server Protocol: Same Omaha protocol as A/B updates.

Language/Technology Stack: - C/C++ (recovery binary), edify scripting language, Java (RecoverySystem API).

License: Apache 2.0 (AOSP)

Strengths: - Simpler than A/B updates. - No need for dual partition slots. - Lower storage requirements. - Well-understood, long history.

Weaknesses: - **DEPRECATED in Android 15.** - Downtime during update (device is unusable in recovery). - Risk of bricking if update is interrupted. - No automatic rollback. - Requires dedicated `/cache` or `/data` space for update package.

Android 15 / RK3588 Compatibility: DEPRECATED. Android 15 CDD effectively mandates Virtual A/B. Not recommended for new designs.

Encapsulatable: Yes — the recovery update flow can be wrapped, but it's legacy.

1.3 A/B (Seamless) Updates System

Architecture Overview: - Introduced in Android 7.0, mandated for new devices since Android 8.0. - Two complete sets of partitions: `system_a`/`system_b`, `boot_a`/`boot_b`, `vendor_a`/`vendor_b`, etc. - Active slot is booted; update is written to inactive slot in background.

Key Components: - **boot_control HAL:** Manages slot selection, marks slots as bootable/unbootable. - **update_engine:** Background daemon that applies updates. - **Slot suffixes:** `_a` and `_b` appended to partition names. - **misc partition:** Stores boot control metadata (active slot, retry count, boot state).

Partition Layout:

```
boot_a, boot_b
system_a, system_b
vendor_a, vendor_b
dtbo_a, dtbo_b
vbmeta_a, vbmeta_b
init_boot_a, init_boot_b  (Android 13+)
misc (boot control info)
```

Update Flow: 1. update_engine checks for update → downloads payload → writes to inactive slot. 2. Post-install verification (mount and check). 3. Mark inactive slot as active via boot_control HAL. 4. Reboot into new slot. 5. If boot fails N times, bootloader rolls back to old slot.

Verification: Same as update_engine (SHA-256, RSA signatures, dm-verity, post-install mount check).

Rollback: Automatic via bootloader — if new slot fails to boot within retry count, old slot is used.

Storage Overhead: ~2x for all A/B-protected partitions (significant on devices with limited storage).

Strengths: - Seamless, no-downtime updates. - Automatic rollback. - Reliable and battle-tested.

Weaknesses: - High storage overhead (dual partitions). - Flash wear from writing entire partitions. - Complexity in partition management.

Android 15 / RK3588 Compatibility: Supported, but Virtual A/B is the current standard.

Encapsulatable: The A/B mechanism is hardware/bootloader-level; can be encapsulated by a higher-level OTA system.

1.4 Virtual A/B Updates

Architecture Overview: - Introduced in Android 11, Android's **main update mechanism** as of Android 13+. - Builds on A/B updates but uses **snapshot-based merging** via dm-snapshot instead of requiring dedicated B partitions. - Uses `super` partition with dynamic partitions — COW (Copy-on-Write) snapshots during update. - Non-A/B is deprecated in Android 15.

Key Innovation: - Instead of having physical `_b` partitions, Virtual A/B uses the `super` partition's free space + COW snapshots. - During update, changes are written to COW snapshots. On reboot, the new system boots from snapshots. - After successful boot, snapshots are **merged** into the actual partitions in the background (merge can be interrupted and resumed).

Components: - **snapshotctl:** Manages COW snapshots. - **update_engine:** Same daemon, with Virtual A/B support. - **dm-snapshot:** Device-mapper snapshot target in Linux kernel. - **libsnapshot:** Userspace library for snapshot management. - **super partition:** Contains dynamic partitions (system, vendor, product, odm, etc.).

Update Flow: 1. update_engine creates COW snapshots for each partition being updated. 2. Update is written into snapshots (not touching live data). 3. On reboot, device boots from snapshot. 4. `snapshotctl` merges snapshots in background after successful boot. 5. If merge is interrupted, it resumes on next boot. 6. If boot fails, bootloader falls back — snapshots are discarded.

Storage Overhead: Much less than legacy A/B — only needs COW space for changed blocks, not full duplicate partitions. Typically 30-50% of A/B overhead.

Verification: Same mechanisms as A/B (SHA-256, RSA, dm-verity, post-install).

Rollback: Automatic. Failed boot → snapshots discarded → old system intact.

Strengths: - Lower storage overhead than legacy A/B. - Same seamless update experience. - Mandatory in Android 13+ (effectively). - Better for devices with limited storage.

Weaknesses: - More complex implementation (dm-snapshot, COW, merge logic). - Merge phase can be slow on large partitions. - Requires kernel support for dm-snapshot and dm-user. - More complex boot flow.

Android 15 / RK3588 Compatibility: This is the required mechanism for Android 15. Must be implemented or adapted for RK3588.

Encapsulatable: The update_engine + Virtual A/B mechanism can be driven by a custom server. The payload format and merge logic must be preserved.

1.5 Google's OTA Infrastructure

Architecture Overview: - Google operates the server-side infrastructure for Pixel and GMS devices. - Uses the **Omaha Protocol** for update checks and delivery. - Update payloads hosted on Google's CDN.

Key Components: - **Omaha Server:** Update check service (XML-based request/response). - **Payload Server:** Hosts OTA payloads for download. - **Rollout Service:** Manages phased deployment percentages. - **CUPS (Chrome OS Update Server):** Separate server for ChromeOS.

Client-Server Protocol: - Omaha Protocol v3 (XML over HTTPS). - Request: app_id, version, hardware_class, delta_ok, update_check. - Response: status, urls, manifest, packages (with hash, size).

Phased Deployment: - Server-controlled rollout percentages. - Device randomness determines which cohort a device falls into. - Can pause/resume rollouts.

License: Proprietary (server-side); Client is Apache 2.0 in AOSP.

Scalability: Billions of devices globally; uses Google's CDN infrastructure.

Strengths: - Battle-tested at unprecedented scale. - Phased rollout support. - Delta updates for bandwidth efficiency. - Tight integration with Android security model.

Weaknesses: - Server-side is proprietary and not available for custom deployments. - Omaha protocol is verbose (XML). - Limited customization options for OEMs.

Android 15 / RK3588 Compatibility: Server not available for custom use; must build custom server or use alternative.

Encapsulatable: The Omaha client in AOSP can be redirected to a custom server by changing the update URL.

1.6 LineageOS OTA System

Architecture Overview: - Simple REST API server that provides update metadata. - Client is the built-in Updater app in LineageOS ROM. - No phased rollout, no delta updates (traditionally), simple full-update model.

Key Components & Data Flow: 1. Client (Updater app) sends POST request to OTA server with device info. 2. Server responds with JSON containing update metadata (URL, MD5, filename, timestamp, channel). 3. Client downloads the full ROM zip. 4. User applies update through recovery or the Updater app.

API Format:

```
{
  "id": null,
  "result": [{
    "incremental": "7f96568932",
    "api_level": 19,
    "url": "http://download.lineageos.org/update.zip",
    "timestamp": "1391086037",
    "md5sum": "4e0a335b378035d12cb6626b6623072b",
    "changes": "http://download.lineageos.org/CHANGES.txt",
    "channel": "nightly",
    "filename": "lineage-21-20240130-NIGHTLY-device.zip"
  }],
  "error": null
}
```

Server Configuration: - Set `cm.updater.uri` in `build.prop` to point to custom OTA server URL. - Or override `conf_update_server_url_def` in `values.xml`.

Update Verification: MD5 checksum only (no cryptographic signatures beyond standard Android package verification).

Rollback: Not supported — manual reflash required.

Rollout: Not supported — all devices see all updates simultaneously.

Language/Technology Stack: - Server: PHP or Node.js (community implementations). - Client: Java/Android (Updater app in AOSP/LineageOS).

License: Apache 2.0 / GPLv2 (various components)

Scalability: Simple; limited by CDN capacity. No server-side device management.

Strengths: - Extremely simple to implement and deploy. - Works with any custom ROM. - Open-source server implementations available. - Easy to self-host.

Weaknesses: - No phased rollout. - No delta updates. - No rollback support. - MD5-only verification (weak). - No device management or inventory. - Recovery-mode only (downtime during update).

Android 15 / RK3588 Compatibility: Can work but doesn't leverage A/B or Virtual A/B. Would need significant enhancement.

Encapsulatable: Very easily encapsulated — it's just a REST API + download URL.

---

2. Embedded/IoT OTA Systems

2.1 Mender.io

Architecture Overview: - End-to-end, client-server OTA update platform for embedded Linux and IoT devices. - Supports full system updates (rootfs), application updates, and container updates. - Enterprise and open-source editions.

Key Components: - **Mender Client (mender):** Runs on device, communicates with server, applies updates. - Written in Go. - Uses A/B partition scheme for rootfs updates. - State machine: Idle → Check Update → Download → Install → Reboot → Commit → Idle. - **Mender Server:** Web-based management UI + REST API. - Device inventory, deployment management, reporting. - Supports phased rollouts with percentage-based deployment. - **Mender Artifact:** Custom update artifact format (.mender) containing the update payload and metadata. - **Mender Connect:** Device-side component for troubleshooting and file transfer.

Data Flow: 1. Client polls server (or receives WebSocket notification) for pending deployments. 2. Server provides artifact download URL. 3. Client downloads artifact via HTTPS. 4. Client writes rootfs to inactive partition. 5. Reboot into new partition. 6. If successful, client sends commit signal → server marks deployment successful. 7. If failed (no commit within timeout), bootloader rolls back to old partition.

Update Verification: - RSA-2048/4096 signature verification on artifacts. - SHA-256 checksums on artifact contents. - TLS for transport security.

Rollback: - Automatic A/B rollback: if new partition doesn't boot or doesn't commit, bootloader reverts. - Configurable commit timeout.

Rollout/Phased Deployment: - Full phased rollout support in Enterprise edition. - Percentage-based deployment (e.g., 10% → 50% → 100%). - Pause/resume capability. - Per-device-group deployment.

Client-Server Protocol: - REST API over HTTPS (JSON). - Device authentication via JWT tokens. - Polling model (default: every 30 minutes) or WebSocket push.

Language/Technology Stack: - Client: Go - Server: Go (backend), JavaScript/React (frontend) - Artifact: Custom format (tar.gz with manifest and signatures) - Integrations: Yocto, Buildroot, Debian, Ubuntu Core

License: - Client: Apache 2.0 / MIT - Server: Apache 2.0 (open-source) + Enterprise (commercial) - Enterprise features: RBAC, phased rollout, dual hosting, compliance reporting

Scalability: - Tested with 10,000+ devices in open-source. - Enterprise supports 100,000+ devices. - Horizontally scalable server architecture.

Strengths: - Mature, well-documented, production-proven (1M+ devices). - End-to-end solution (client + server + UI). - A/B updates with automatic rollback. - Artifact signing and verification. - Yocto integration is excellent (meta-mender). - Phased rollout support (Enterprise). - Application and container updates in addition to system updates. - Active community and commercial support. - Microcontroller (MCU) support added recently.

Weaknesses: - Enterprise features (phased rollout, RBAC) are paywalled. - Go client may be heavy for very resource-constrained devices. - Artifact format is Mender-specific (not standard). - Limited delta/differential update support (Enterprise only, and limited). - Requires dual rootfs partitions (A/B). - Server setup requires Docker or Kubernetes.

Android 15 / RK3588 Compatibility: - **Partial.** Mender is designed for Linux, not Android. Could theoretically manage the Linux kernel and rootfs on an Android device, but would conflict with Android's own update mechanisms. - Not a natural fit for Android 15's Virtual A/B system. - Could be used for non-Android Linux on RK3588.

Encapsulatable: Yes — Mender's client-server protocol is well-documented REST API. A custom client or server could be built to interoperate.

2.2 RAUC (Robust Auto-Update Controller)

Architecture Overview: - Lightweight, flexible update client for embedded Linux devices. - Focuses on the device-side update process; server-agnostic. - Designed by Pengutronix for industrial and automotive applications.

Key Components: - **RAUC Client (rauc):** The update daemon running on the device. - C-based, ~512KB binary — very lightweight. - D-Bus API for integration with applications. - State machine-based update process. - **RAUC Bundle:** Update artifact format (.raucb) — a SquashFS image containing: - Rootfs images (or other payloads). - manifest.raucm (JSON manifest). - Cryptographic signatures. - **RAUC Hawkbit:** Optional integration with Eclipse hawkBit for fleet management.

Data Flow: 1. RAUC receives update bundle (via download, USB, or external trigger). 2. Verifies bundle signature (X.509). 3. Determines target slot (active vs inactive in A/B scheme). 4. Writes image to inactive slot. 5. Updates bootloader environment to boot from new slot. 6. Reboots into new system. 7. On successful boot, marks new slot as good.

Update Verification: - X.509 certificate-based signature verification (asymmetric cryptography). - SHA-256 hash verification of bundle contents. - Optional per-recipient encryption (asymmetric). - HTTP(S) streaming support (install without intermediate storage).

Rollback: - A/B partition scheme with automatic rollback via bootloader. - Configurable boot count — if new slot fails N times, reverts. - Custom hooks for pre/post-install verification.

Rollout/Phased Deployment: - Not built into RAUC itself — requires hawkBit integration for fleet management. - RAUC + hawkBit provides full phased rollout support.

Client-Server Protocol: - RAUC itself is server-agnostic. - D-Bus API on the device side. - hawkBit integration uses hawkBit's DDI API (REST over HTTP).

Language/Technology Stack: - Client: C (GLib-based) - Build integration: Yocto (meta-rauc), Buildroot, PTXdist - Bundle format: SquashFS + JSON manifest - Cryptography: OpenSSL

License: LGPL-2.1

Scalability: Client is extremely lightweight. Fleet scalability depends on server (hawkBit or custom).

Strengths: - Very lightweight client (~512KB). - Clean, well-designed architecture. - X.509-based signing (industry standard). - Flexible — supports various partition layouts and update strategies. - Not tied to any specific server — server-agnostic. - Excellent Yocto integration (meta-rauc). - D-Bus API for application integration. - Supports A/B, recovery, and custom partition schemes. - HTTP(S) streaming (no intermediate storage needed). - Per-recipient encryption. - Active development by Pengutronix (commercial + community). - Clean architecture for complex scenarios.

Weaknesses: - No built-in server or fleet management UI. - Requires external server (hawkBit or custom) for fleet management. - No built-in delta update support. - Smaller community than Mender. - Less documentation compared to Mender.

Android 15 / RK3588 Compatibility: - Not designed for Android. Could potentially work with Linux on RK3588. - Would need significant adaptation for Android's Virtual A/B mechanism.

Encapsulatable: Very encapsulatable. RAUC is designed to be server-agnostic and can be integrated into any OTA infrastructure.

2.3 SWUpdate

Architecture Overview: - Highly configurable and extensible update framework for embedded Linux. - Supports multiple update strategies: A/B, recovery, single-copy with fallback. - Integrated web server for local updates + hawkBit backend for fleet deployments.

Key Components: - **SWUpdate Client:** Core daemon running on device. - C-based with Lua scripting support. - Parser-based architecture: different parsers handle different image formats. - Supports multiple handlers: raw, ubivol, CFI (flash), ext4,

etc. - **SWUpdate Web Server:** Built-in web server for local (SOHO router-style) updates. - **hawkBit Integration:** For fleet deployment via Eclipse hawkBit. - **SWU Image Format:** Custom .swu format (CPIO archive with sw-description manifest).

Data Flow: 1. SWUpdate receives update image (download, USB, or from hawkBit). 2. Parses sw-description to determine what to install and where. 3. Verifies signature (RSA or CMS). 4. Writes images to target partitions/storage. 5. Updates bootloader environment. 6. Reboots into new system.

Update Verification: - RSA or CMS signature verification. - SHA-256 hash verification. - Hardware revocation list support. - Custom verification scripts (Lua).

Rollback: - A/B partition support with bootloader-based rollback. - Recovery partition support. - Configurable post-install verification scripts.

Rollout/Phased Deployment: - Via hawkBit integration only. - No built-in phased rollout.

Client-Server Protocol: - hawkBit DDI API for fleet management. - Built-in web server for local updates. - Custom REST API endpoints.

Language/Technology Stack: - Client: C (with Lua scripting) - Image format: CPIO archive + sw-description (JSON/libconfig) - Build: Yocto, Buildroot - Crypto: OpenSSL/mbedTLS/wolfSSL

License: GPL-2.0 (or later)

Scalability: Depends on backend (hawkBit). Client is lightweight.

Strengths: - Very flexible and configurable. - Multiple update strategies supported. - Built-in web server for local updates. - Lua scripting for custom install logic. - Supports many storage types (eMMC, NAND, NOR, SPI, UBI, etc.). - Good hawkBit integration. - Active community. - Supports hardware compability checking.

Weaknesses: - GPL license may be restrictive for some commercial products. - Configuration is complex (sw-description format). - No built-in server/fleet management UI. - Less polished than Mender. - Security model is less robust than RAUC’s X.509 approach.

Android 15 / RK3588 Compatibility: Linux-only; not designed for Android. Could work for Linux on RK3588.

Encapsulatable: Yes, especially with its modular handler architecture.

2.4 Eclipse hawkBit

Architecture Overview: - Open-source software update server for IoT device fleets. - Provides the management/backend layer; works with various update clients (SWUpdate, RAUC, Mender, custom). - Eclipse Foundation project.

Key Components: - **hawkBit Server:** Spring Boot-based Java application. - Management UI (Angular-based web interface). - REST API (DDI - Device Direct Integration, MGMT - Management API). - Rollout management with phased deployment. - **hawkBit Device Simulator:** For testing. - **Update Clients:** Not provided by hawkBit — uses existing clients (RAUC, SWUpdate, etc.).

Data Flow: 1. Devices poll hawkBit DDI API for pending updates. 2. hawkBit returns update metadata (URLs, hashes, size). 3. Device downloads artifacts from configured artifact repository (e.g., S3, local file server). 4. Device applies update using its native update mechanism. 5. Device reports status back to hawkBit. 6. hawkBit tracks deployment progress per device.

Update Verification: Server-side only — hawkBit verifies artifact integrity at upload time. Device-side verification is delegated to the update client.

Rollback: Not directly managed by hawkBit — delegated to client-side mechanism.

Rollout/Phased Deployment: - **Excellent support.** This is hawkBit's core strength. - Rollout groups with configurable percentages. - Auto-confirmation or manual confirmation per group. - Trigger-based rollouts. - Distribution set management. - Target filter queries for selective deployment.

Client-Server Protocol: - DDI (Device Direct Integration) API: REST over HTTP, JSON. - Polling-based (devices poll for updates). - Standard endpoints: `/controller/v1/{targetId}`. - Management API: REST, JSON, for administrative operations.

Language/Technology Stack: - Server: Java (Spring Boot), Angular (UI) - Database: SQL (MySQL, PostgreSQL, H2) - Artifact storage: File system, S3, Azure Blob

License: EPL-2.0 (Eclipse Public License 2.0)

Scalability: - Designed for fleet management at scale. - Horizontal scaling via Spring Boot and database clustering. - Tested with hundreds of thousands of devices.

Strengths: - Best-in-class phased rollout/rollout management. - Server-only — works with any update client. - Clean DDI API for device integration. - Professional management UI. - Eclipse Foundation governance. - Mature and widely deployed. - Flexible deployment strategies.

Weaknesses: - Java/Spring Boot server is resource-heavy. - No built-in update client — must integrate with RAUC/SWUpdate/custom. - DDI API is polling-based (no push/WebSocket). - Setup complexity (Java, database, etc.). - Less active development than Mender.

Android 15 / RK3588 Compatibility: Could serve as the backend server for a custom Android OTA system. The DDI API could be adapted.

Encapsulatable: Yes — hawkBit is specifically designed to be client-agnostic. Custom Android client could use the DDI API.

2.5 OSTree + rpm-ostree

Architecture Overview: - OSTree (libostree) is a "git for operating system binaries" — content-addressed, versioned filesystem tree management. - rpm-ostree builds on OSTree to provide hybrid image/package updates for RPM-based systems. - Used by Fedora CoreOS, Fedora Silverblue, CentOS Atomic, and others. - **Note:** Development focus is shifting to `bootc` (bootable containers) as the successor.

Key Components: - **OSTree:** Core library for managing versioned filesystem trees. - Content-addressed object store (like git, but for files). - Hard-link based checkout (zero-cost for identical files). - Atomic swap of bootloader target. - **rpm-ostree:** Layer on top of OSTree that adds RPM package support. - Can layer RPMs on top of base OSTree commit. - Hybrid image/package model. - `rpm-ostree install/upgrade/rollback` commands. - **bootc:** Emerging successor, using bootable OCI container images.

Data Flow: 1. `rpm-ostree upgrade` pulls new OSTree commit from remote repository. 2. New commit is deployed as a new deployment (hard-linked). 3. On next boot, bootloader points to new deployment. 4. Old deployment remains available for rollback. 5. Automatic cleanup of old deployments after N deployments.

Update Verification: - GPG signature verification on OSTree commits. - Content-addressed storage ensures integrity (SHA-256 of content). - Static delta updates for efficient bandwidth usage.

Rollback: - Instant: `rpm-ostree rollback` swaps bootloader entry. - Previous deployment is always preserved. - Configurable number of deployments to keep.

Rollout/Phased Deployment: - Not built into OSTree itself. - Fedora uses different update channels (stable, testing, etc.). - Could be implemented with custom infrastructure.

Client-Server Protocol: - OSTree repository protocol (HTTP-based, like git smart HTTP). - Static delta files served over HTTP.

Language/Technology Stack: - C (libostree core) - Python (rpm-ostree compose) - Rust (bootc) - Build: Meson, RPM

License: LGPL-2.1 (OSTree), Apache-2.0 (rpm-ostree)

Scalability: OSTree is highly efficient — delta updates, content deduplication, hard-link based. Server is a static HTTP repository.

Strengths: - Atomic updates with instant rollback. - Content-addressed = deduplication and integrity. - Hybrid image/package model (rpm-ostree). - Static deltas for bandwidth efficiency. - Zero-cost for unchanged files (hard links). - Battle-tested (Fedora CoreOS, Silverblue). - Works well with container-based workflows (bootc).

Weaknesses: - RPM-centric (rpm-ostree); not applicable to non-RPM systems directly. - Read-only /usr filesystem model. - Not designed for Android. - Learning curve for OSTree concepts. - bootc is still maturing. - Limited partial update support (mostly full system updates).

Android 15 / RK3588 Compatibility: Not directly applicable to Android. Could be used for Linux on RK3588 but would need significant adaptation.

Encapsulatable: OSTree is a library that can be integrated. The repository protocol is simple HTTP.

2.6 Foundries.io

Architecture Overview: - IoT device development and management platform. - Based on OSTree for update delivery. - FoundriesFactory: cloud-based CI/CD + OTA platform. - Linux microPlatform (LmP): minimal, OTA-updatable OS for IoT.

Key Components: - **FoundriesFactory:** Cloud service providing CI/CD, OTA, and device management. - **Linux microPlatform (LmP):** Reference OS built with Yocto, updated via OSTree. - **fioctl:** CLI tool for managing factories and devices. - **aktualizr:** OTA client (based on OSTree) running on devices.

Update Verification: - OSTree-based (GPG signatures, content-addressed). - Uptane framework for automotive-grade security. - Hardware root of trust support.

Rollback: OSTree-based instant rollback.

Rollout/Phased Deployment: - Via FoundriesFactory web UI. - Device group management. - Wave-based deployment.

Language/Technology Stack: - C++ (aktualizr client) - Python (fioctl CLI, server) - Yocto (LmP) - OSTree (update delivery)

License: Various open-source (MIT, Apache 2.0) + commercial subscription

Scalability: Cloud-managed; designed for fleet management.

Strengths: - Complete CI/CD to OTA pipeline. - Uptane security framework (automotive grade). - OSTree-based efficient updates. - Good for Yocto-based projects. - MCU support added.

Weaknesses: - Commercial subscription required. - Tied to FoundriesFactory cloud service. - Not designed for Android. - Smaller community.

Android 15 / RK3588 Compatibility: Linux/IoT only; not applicable to Android.

Encapsulatable: Partially — aktualizr is open-source but designed for FoundriesFactory.

2.7 UpdateHub

Architecture Overview: - Cloud-based OTA update platform for embedded Linux devices. - Simplified, managed service approach.

Key Components: - **UpdateHub Cloud:** Managed server with web dashboard. - **UpdateHub Agent:** Device-side client (C-based). - Yocto integration (meta-updatehub).

Update Verification: RSA signature verification on packages.

Rollback: A/B partition with bootloader-based rollback.

Rollout/Phased Deployment: Supported via UpdateHub Cloud dashboard.

Language/Technology Stack: - Agent: C - Server: Cloud-hosted (proprietary) - Integration: Yocto

License: - Agent: MIT - Cloud: Proprietary (freemium model)

Strengths: - Easy to get started. - Managed cloud service. - Good Yocto integration.

Weaknesses: - Vendor lock-in to UpdateHub Cloud. - Limited customization. - Smaller community. - Cloud-dependent.

Android 15 / RK3588 Compatibility: Linux only; not applicable to Android.

Encapsulatable: Limited — tied to UpdateHub Cloud.

2.8 NervesHub

Architecture Overview: - OTA firmware management for Elixir/Nerves devices. - Designed specifically for the Nerves embedded Elixir framework.

Key Components: - **NervesHub Web:** Web dashboard for managing devices and firmware. - **NervesHub Device:** Device-side library (Elixir). - Firmware signed with Ed25519 or ECDSA keys.

Update Verification: Cryptographic signature verification (Ed25519/ECDSA).

Rollback: Not automatic — requires manual reversion to previous firmware.

Rollout/Phased Deployment: Supported via deployment groups.

Language/Technology Stack: - Elixir/Phoenix (server and device) - Nerves framework

License: Apache 2.0 (open-source) + commercial hosting

Scalability: Suitable for small-to-medium fleets; limited by Phoenix/Elixir scalability.

Strengths: - Excellent for Elixir/Nerves ecosystem. - Clean API and web interface. - Firmware signing built-in. - Open-source.

Weaknesses: - Elixir/Nerves ecosystem lock-in. - Not applicable to C/C++ or Android devices. - Small community. - Limited rollback support.

Android 15 / RK3588 Compatibility: Not applicable — Elixir/Nerves only.

Encapsulatable: Limited to Nerves ecosystem.

2.9 EdgeX Foundry

Architecture Overview: - EdgeX Foundry is an **IoT edge computing framework**, not primarily an OTA update system. - It provides a microservices-based middleware for IoT edge computing. - OTA capability is not a core feature — it would need to be integrated via external tools.

Relevance to OTA: Indirect — EdgeX provides the infrastructure for managing IoT devices, but actual OTA updates would be handled by Mender, RAUC, hawkBit, or similar.

License: Apache 2.0

Android 15 / RK3588 Compatibility: Not directly relevant for OTA.

Encapsulatable: N/A for OTA purposes.

2.10 Bosch IoT Rollouts

Architecture Overview: - Commercial OTA update service from Bosch. - Part of the Bosch IoT Suite. - Targets automotive and industrial IoT.

Key Features: - Cloud-managed rollouts. - Firmware, software, and configuration updates. - Campaign-based deployment. - Integration with Bosch IoT Hub and IoT Things.

Update Verification: Cryptographic verification (details proprietary).

Rollback: Supported but details are proprietary.

Rollout/Phased Deployment: Full campaign-based rollout with targeting.

Language/Technology Stack: Proprietary (Java-based backend likely).

License: Proprietary / Commercial

Scalability: Enterprise-grade; designed for automotive scale.

Strengths: - Automotive-grade security and compliance. - Part of broader Bosch IoT Suite. - Campaign-based rollout. - Professional support.

Weaknesses: - Proprietary and expensive. - Vendor lock-in. - Limited public documentation. - Not open-source.

Android 15 / RK3588 Compatibility: Not specifically designed for Android. Could potentially manage Linux devices on RK3588.

Encapsulatable: No — proprietary service.

---

3. Linux Distribution OTA

3.1 Fedora Silverblue / OSTree

Architecture Overview: - Fedora's immutable desktop OS using OSTree for atomic system updates. - Read-only /usr, with /etc and /var writable. - Flatpak for application delivery. - Toolbox for development containers.

Update Mechanism: - `rpm-ostree upgrade` pulls new OSTree commit. - Hard-link based deployment — identical files are zero-cost. - Atomic swap at boot time via bootloader configuration. - Static delta updates for bandwidth efficiency.

Rollback: `rpm-ostree rollback` — instant, previous deployment always preserved.

Rollout: Different branches/channels (stable, testing, updates-testing).

License: LGPL-2.1 (OSTree), Apache-2.0 (rpm-ostree), various Fedora licenses

Android 15 / RK3588 Compatibility: Linux-only. Could run on RK3588 as Linux but not for Android.

Encapsulatable: OSTree is a library; the protocol is simple HTTP. Could be adapted.

3.2 Ubuntu Core / Snap Updates

Architecture Overview: - Ubuntu Core is an immutable, all-snap OS for IoT. - Every component (kernel, OS, apps) is a snap package. - Snaps are SquashFS images with metadata and signatures.

Update Mechanism: - `snapd` daemon checks Snap Store periodically for updates. - Automatic background download and installation. - Snap-delta format for efficient OTA updates (50-90% size reduction in Ubuntu Core 26). - Channel-based: stable, candidate, beta, edge.

Update Verification: - Snap assertions: cryptographic signatures verifying snap authenticity. - Snap store is the central authority. - Model assertion ties device to authorized snaps.

Rollback: - Automatic: if new snap fails to start, snapd reverts to previous revision. - `snap revert` for manual rollback. - Up to 3 revisions kept by default.

Rollout/Phased Deployment: - Channel-based: promote from edge → beta → candidate → stable. - Snap Store controls which devices see which versions. - Progressive releases (beta feature).

Language/Technology Stack: - snapd: Go - Snaps: SquashFS + JSON metadata + assertions - Store: Proprietary (Canonical) - Client-server: Custom REST API over HTTPS

License: - snapd: GPL-3.0 - Ubuntu Core: Various open-source + proprietary bits - Snap Store: Proprietary (Canonical)

Scalability: Canonical manages the Snap Store for millions of devices.

Strengths: - Complete, integrated solution. - Automatic rollback. - Delta updates (snap-delta) are very efficient. - Channel-based deployment. - Transactional updates. - Security model with assertions. - Ubuntu Core 26 introduces much smaller updates.

Weaknesses: - Snap Store is proprietary (Canonical-controlled). - Vendor lock-in to Canonical ecosystem. - snapd is GPL-3.0 (copyleft concerns). - Slow startup times for snaps (SquashFS mount overhead). - Controversial in the Linux community. - Not designed for Android. - Requires Ubuntu Core OS.

Android 15 / RK3588 Compatibility: Not applicable to Android. Could run Ubuntu Core on RK3588 but not for Android OTA.

Encapsulatable: Limited — tightly coupled to snapd and Snap Store.

3.3 ChromeOS Update Mechanism

Architecture Overview: - Uses the same `update_engine` as Android (shared codebase). - A/B partition scheme with USR-A and USR-B partitions. - Automatic background updates. - Update check at boot and periodically.

Key Differences from Android: - ChromeOS uses a simpler partition layout: `ROOT-A`, `ROOT-B`, `KERNEL-A`, `KERNEL-B`. - Update is applied to inactive partition, verified, then marked as active. - Sign-out (not reboot) can trigger update application. - P2P update sharing on local networks. - Enterprise policy for update management.

Update Flow: 1. `update_engine` checks Omaha server for updates. 2. Downloads payload in background. 3. Writes to inactive partition. 4. Post-install verification. 5. On next sign-out/reboot, new partition is used. 6. If fails, automatic rollback.

Verification: Same as Android (SHA-256, RSA signatures).

Rollback: Automatic via partition scheme.

Language/Technology Stack: C++ (update_engine), Omaha protocol.

License: Chromium OS is BSD-3-Clause / Apache 2.0; ChromeOS is proprietary.

Android 15 / RK3588 Compatibility: Same update_engine as Android. Not directly applicable to RK3588 Android.

Encapsulatable: Same as Android update_engine.

3.4 CoreOS / Flatcar Container Linux Update

Architecture Overview: - Container-optimized OS with automatic, atomic updates. - Uses A/B partition scheme: USR-A and USR-B. - `update_engine` (same as ChromeOS/Android) for applying updates. - `locksmith` or `Nebraska` for reboot management.

Key Components: - **update_engine:** Downloads and applies OS updates (same as ChromeOS). - **locksmith:** Reboot strategy manager (older, deprecated). - **Nebraska:** Modern update manager (Flatcar). - **update-ctrl:** Manual update control.

Partition Layout:

```
USR-A, USR-B (read-only root filesystem)
boot-A, boot-B (kernel + bootloader config)
OEM partition (cloud-config)
```

Update Strategies: - **Off:** No automatic updates. - **Reboot:** Update immediately on detection. - **Best-effort:** Update when no active containers (default). - **Container-runtime:** Coordinate with container runtime.

Verification: Same as ChromeOS (update_engine with Omaha protocol).

Rollback: Automatic — if new USR partition doesn't boot, fall back to old.

Language/Technology Stack: C++ (update_engine), Go (Nebraska), Shell (locksmith).

License: Apache 2.0 (Flatcar); CoreOS was Apache 2.0 (now Fedora CoreOS).

Scalability: Designed for server fleets; Kubernetes integration.

Strengths: - Automatic, transparent updates. - Battle-tested in container orchestration environments. - Multiple update strategies for different workloads. - A/B rollback protection. - Simple, container-optimized design.

Weaknesses: - Container-specific; not general-purpose. - Requires container workflow. - Limited flexibility for non-container use cases.

Android 15 / RK3588 Compatibility: Not applicable to Android.

Encapsulatable: update_engine is the same as Android/ChromeOS.

3.5 openSUSE MicroOS

Architecture Overview: - Transactional update system using Btrfs snapshots. - Read-only root filesystem. - `transactional-update` command for atomic system updates.

Update Mechanism: 1. `transactional-update` creates a new Btrfs snapshot. 2. Updates are applied to the new snapshot. 3. On next boot, system boots into the new snapshot. 4. If anything fails, boot into previous snapshot (via GRUB).

Rollback: Instant via Btrfs snapshot selection in GRUB bootloader.

Rollout: Not built-in; relies on openSUSE release channels.

Language/Technology Stack: Btrfs, Bash (transactional-update), Zypper (package manager).

License: Various open-source (GPL, etc.)

Strengths: - Btrfs-based snapshots are very efficient. - Instant rollback via bootloader. - Works with RPM packages (Zypper). - Simple concept.

Weaknesses: - Requires Btrfs. - Not designed for embedded/Android. - Limited delta update support. - Rollback is manual (user must select in GRUB).

Android 15 / RK3588 Compatibility: Not applicable to Android.

Encapsulatable: The Btrfs snapshot concept could be adapted but not directly.

3.6 NixOS Atomic Updates

Architecture Overview: - NixOS uses the Nix package manager's unique approach to system configuration. - Each system configuration is a "profile generation" — a complete, immutable filesystem tree in `/nix/store`. - `/run/current-system` symlink points to the active generation.

Update Mechanism: 1. `nixos-rebuild switch` builds a new system configuration. 2. New generation is created in `/nix/store` (content-addressed, deduplicated). 3. Symlink is atomically updated to point to new generation. 4. GRUB entry is added for the new generation. 5. Reboot into new generation.

Rollback: - `nixos-rebuild switch --rollback` — instant, previous generation is always preserved. - GRUB menu lists all previous generations. - Generations can be garbage-collected after N days.

Verification: NAR hash verification (SHA-256 of every path in the store). Signed channels.

Rollout: Not built-in; uses Nix channels (stable, unstable).

Language/Technology Stack: Nix language, C++ (nix daemon), Bash.

License: LGPL-2.1 (Nix)

Strengths: - Truly atomic — entire system is a single immutable snapshot. - Perfect rollback — every previous generation is preserved. - Content-addressed = automatic deduplication. - Reproducible builds. - Very fine-grained control.

Weaknesses: - Requires Nix ecosystem (steep learning curve). - Large storage requirements (/nix/store grows). - Not designed for embedded/Android. - Complex setup. - Not bandwidth-efficient for remote updates (large closure transfers).

Android 15 / RK3588 Compatibility: Not applicable to Android.

Encapsulatable: The concept is powerful but the Nix ecosystem is hard to extract.

---

4. Windows OTA

4.1 Windows Update for Business (WUfB)

Architecture Overview: - Microsoft's cloud-based update management service. - Integrated with Microsoft Intune and Azure AD. - Manages Windows 10/11 quality and feature updates.

Key Features: - Deployment rings (pilot, broad, etc.). - Update deferral policies. - Approval workflows. - Driver and firmware updates. - Integration with Microsoft Endpoint Manager.

Verification: Microsoft digital signatures, SHA-2 code signing.

Rollback: Limited — Windows can uninstall some updates within 10 days (feature updates). Quality updates can be uninstalled individually.

Rollout: Ring-based deployment with configurable delays.

Language/Technology Stack: Proprietary (Microsoft), WSUS protocol, BITS (Background Intelligent Transfer Service).

License: Proprietary (requires Windows licensing)

Strengths: - Integrated with Microsoft ecosystem. - Ring-based deployment. - Cloud-managed. - Enterprise-grade support.

Weaknesses: - Windows-only. - Proprietary. - Requires Microsoft licensing. - Limited control over update content.

Android 15 / RK3588 Compatibility: Not applicable.

Encapsulatable: No.

4.2 WSUS (Windows Server Update Services)

Architecture Overview: - On-premises update server for Windows devices. - Downloads updates from Microsoft Update and distributes to internal devices. - Approve/decline individual updates.

Key Features: - Local update repository. - Update approval workflow. - Computer group targeting. - Reporting.

License: Free with Windows Server

Status: Being deprecated — Microsoft is pushing toward WUfB and Intune.

Android 15 / RK3588 Compatibility: Not applicable.

Encapsulatable: No.

4.3 Open-Source Windows Update Systems

- **No significant open-source Windows update system exists.**
- WSUS alternatives are typically commercial patch management tools (Automox, ManageEngine, etc.).
- `kbupdate` (PowerShell module) for offline patching — not a full OTA system.
- The Windows update protocol is proprietary and undocumented.

---

5. Commercial Solutions

5.1 AWS IoT Device Management

Architecture Overview: - AWS service for managing IoT device fleets. - OTA update capability through AWS IoT Jobs. - Integrates with AWS IoT Core for device connectivity.

Key Features: - **AWS IoT Jobs:** Define and deploy update jobs to device fleets. - **Fleet indexing:** Query and filter devices for targeted updates. - **Secure tunneling:** Remote access to devices. - **Device Defender:** Security auditing. - **Code signing:** AWS Signer for firmware signing.

Update Flow: 1. Create OTA update in AWS IoT console/API. 2. AWS IoT Jobs notifies devices via MQTT. 3. Device downloads firmware from S3 (pre-signed URL). 4. Device applies update. 5. Device reports status via MQTT.

Update Verification: AWS Code Signer (RSA/ECDSA), S3 integrity checks.

Rollback: Manual — deploy previous version as a new job.

Rollout: Job rollouts with configurable concurrency and rate.

Language/Technology Stack: MQTT, HTTPS, AWS SDK (C, Python, etc.), CloudFormation/Terraform.

License: Proprietary (AWS service), pay-per-use pricing.

Scalability: AWS-scale (millions of devices).

Strengths: - AWS ecosystem integration. - S3 + CloudFront for global delivery. - MQTT-based push notifications. - Fine-grained IAM access control. - Code signing integration. - Fleet indexing for targeted updates.

Weaknesses: - AWS lock-in. - Complex pricing model. - No built-in A/B update mechanism — must be implemented on device. - MQTT requires persistent connection or periodic polling. - Limited device-side update orchestration.

Android 15 / RK3588 Compatibility: Could serve as the backend for a custom OTA system. Device-side client would need to be custom-built.

Encapsulatable: The AWS IoT Jobs protocol can be used as a transport layer; device-side logic is custom.

5.2 Azure IoT Hub Device Update

Architecture Overview: - Azure service for deploying OTA updates to IoT devices. - Supports image-based, package-based, and script-based updates. - Integrates with Azure IoT Hub for device connectivity.

Key Features: - **Device Update Agent:** Open-source C-based agent running on devices. - **Update Manifest:** JSON document describing the update (files, hashes, install steps). - **Import Manifest:** Describes how to ingest updates. - **Deployment groups:** Targeted rollouts. - **Delta updates:** Support for reducing download size.

Update Flow: 1. Import update into Device Update service. 2. Create deployment targeting a device group. 3. Device Update Agent receives notification via IoT Hub. 4. Agent downloads update content from Azure Blob Storage. 5. Agent applies update (via handler: swupdate, script, or custom). 6. Agent reports progress and result.

Update Verification: SHA-256 hash verification, optional code signing.

Rollback: Deploy previous version as a new update; no automatic rollback.

Rollout: Group-based deployment with percentage-based rollout.

Language/Technology Stack: - Agent: C (open-source) - Server: Azure (proprietary) - Protocol: MQTT/AMQP/HTTPS via IoT Hub

License: Agent is MIT; Service is proprietary (Azure pricing).

Scalability: Azure-scale.

Strengths: - Open-source device agent. - Azure ecosystem integration. - Multiple update types (image, package, script). - Delta update support. - Group-based deployment.

Weaknesses: - Azure lock-in. - Device agent is relatively new and less mature. - No built-in A/B mechanism. - Complex Azure pricing.

Android 15 / RK3588 Compatibility: The open-source agent could potentially be adapted for Android on RK3588, but would need significant work.

Encapsulatable: The agent is open-source (MIT) and can be modified. The protocol could be adapted.

5.3 JFrog Connect (formerly Upswift)

Architecture Overview: - Commercial IoT device management and OTA update platform. - Now part of JFrog (artifactory ecosystem). - Supports Linux-based IoT devices.

Key Features: - Full system updates (rootfs). - Application/container updates. - File-based updates. - Device monitoring and commands. - Rollback support.

Update Verification: File hash verification.

Rollback: Supported — previous version can be redeployed.

Rollout: Group-based deployment.

Language/Technology Stack: Proprietary agent, JFrog Artifactory backend.

License: Proprietary / Commercial

Scalability: Enterprise-grade via JFrog infrastructure.

Strengths: - JFrog Artifactory integration for artifact management. - Multiple update types. - Good for software supply chain management. - Commercial support.

Weaknesses: - Proprietary and commercial. - JFrog ecosystem lock-in. - Limited public documentation. - Not designed for Android.

Android 15 / RK3588 Compatibility: Linux/IoT only; not for Android.

Encapsulatable: No — proprietary.

5.4 Memfault

Architecture Overview: - Embedded observability platform that added OTA update capabilities. - Primarily focused on device monitoring, debugging, and fleet analytics. - OTA is a newer feature, added for both MCU and embedded Linux.

Key Features: - **OTA for MCU:** Firmware updates for microcontrollers. - **OTA for Embedded Linux:** Full system updates for Linux devices. - **Android OTA:** Custom OTA client for Android devices using Memfault's service. - Fleet-wide deployment with software version targeting. - Real-time monitoring of update progress. - Integration with Memfault's observability (crash reporting, metrics).

Update Verification: Cryptographic verification (device-specific).

Rollback: Device-side A/B mechanism for Linux; chip-specific for MCU.

Rollout: Version-based targeting and phased deployment.

Language/Technology Stack: - SDK: C (MCU), Go (Linux), Java/Kotlin (Android) - Server: Proprietary (Memfault cloud) - Protocol: REST API over HTTPS

License: SDK is MIT/BSD; Cloud service is proprietary/commercial.

Scalability: Cloud-managed; designed for fleet scale.

Strengths: - Integrated observability (unique — OTA + monitoring + debugging). - Android OTA client available. - MCU support. - Real-time update monitoring. - Good developer experience.

Weaknesses: - Commercial (free tier limited). - OTA is not the primary focus — monitoring is. - Less mature OTA than Mender or RAUC. - Cloud-dependent. - Limited customization of update flow.

Android 15 / RK3588 Compatibility: Has an Android OTA client. Could potentially work with RK3588 Android, but would need evaluation.

Encapsulatable: The Android SDK is available and could be integrated.

5.5 Qualcomm FOTA

Architecture Overview: - Qualcomm's Firmware Over-The-Air solution for devices using Qualcomm chipsets. - Integrated into Qualcomm's device management ecosystem. - Targets smartphones, IoT, and automotive.

Key Features: - Full firmware update (bootloader, modem, system). - Delta updates for bandwidth efficiency. - Hardware-backed security (Qualcomm Secure Boot). - Integration with Qualcomm's Snapdragon platforms.

Update Verification: Qualcomm Secure Boot chain, hardware root of trust.

Rollback: Qualcomm's A/B implementation with rollback.

License: Proprietary / Commercial (Qualcomm licensing required)

Scalability: Carrier-scale.

Strengths: - Deep hardware integration. - Hardware-backed security. - Delta updates. - Carrier-grade.

Weaknesses: - Qualcomm-only. - Proprietary and expensive. - Requires Qualcomm licensing. - Not applicable to RK3588.

Android 15 / RK3588 Compatibility: Not applicable — Qualcomm-specific.

Encapsulatable: No — tied to Qualcomm hardware.

5.6 Applistent

Architecture Overview: - Limited public information available. - Appears to be a Japanese company providing OTA solutions. - Primarily for automotive and mobile devices.

Key Features: (based on limited public info) - Firmware OTA for embedded devices. - Automotive focus.

License: Proprietary / Commercial

Android 15 / RK3588 Compatibility: Unknown; likely not applicable.

Encapsulatable: Unknown.

---

6. Android 15 / RK3588 Specifics

6.1 What Changed in Android 15 for OTA

Key Changes: 1. **Non-A/B Updates Deprecated:** Android 15 CDD effectively deprecates non-A/B (recovery-based) updates. Virtual A/B is the mandated update mechanism for new devices. 2. **Virtual A/B Enhancements:** Improved snapshot merge performance and reliability. 3. **16KB Page Size Support:** Android 15 introduces support for 16KB page sizes, which affects OTA payload generation and partition alignment. 4. **Improved Post-Install Verification:** Better checking of updated partitions. 5. **Privacy Sandbox Integration:** Some OTA-related privacy improvements. 6. **Kernel 6.6 GKI:** Android 15 uses GKI kernel 6.6, which affects boot images and OTA payload structure. 7. **init_boot Partition:** The generic ramdisk is moved to `init_boot` partition (introduced in Android 13), meaning OTA must update this partition as well.

Impact on OTA Design: - Must use Virtual A/B or at minimum legacy A/B. - Recovery-only OTA is not viable for new devices. - Payload must include all dynamic partitions in super partition. - Must handle 16KB page alignment.

6.2 Orange Pi 5 Max Android 15 Specifics

Hardware: - Rockchip RK3588: 4x Cortex-A76 @ 2.4GHz + 4x Cortex-A55 @ 1.8GHz - Mali-G610 GPU - 16GB LPDDR5 RAM - eMMC 5.1 + NVMe SSD storage - Gigabit Ethernet + Wi-Fi 6

Android on Orange Pi 5 Max: - Community builds exist (Android 14/15). - Rockchip provides Android BSP with RK3588 support. - Partition layout is Rockchip-specific (not standard AOSP). - Uses Rockchip's parameter.txt for partition definition. - Default update mechanism is Rockchip's recovery-based OTA (not A/B).

Challenges: - Rockchip BSP defaults to non-A/B layout. - Enabling A/B requires modifying parameter.txt and rebuilding. - Bootloader (U-Boot with Rockchip patches) must support A/B slot selection. - Community Android builds may not support Virtual A/B.

6.3 A/B Partition Layout for RK3588

Standard Rockchip Partition Layout (Non-A/B):

```
0x00002000@0x00004000(uboot)
0x00002000@0x00006000(misc)
0x00020000@0x00008000(boot)
0x00040000@0x00028000(recovery)
0x00010000@0x00068000(backup)
-@0x00078000(rootfs:grow)
```

A/B Partition Layout for RK3588 (Modified):

```
0x00002000@0x00004000(uboot)
0x00002000@0x00006000(misc)
0x00020000@0x00008000(boot_a)
0x00020000@0x00028000(boot_b)
0x00040000@0x00048000(system_a)   -- or via super partition
0x00040000@0x00088000(system_b)
0x00010000@0x000c8000(vendor_a)
0x00010000@0x000d8000(vendor_b)
-@0x000e8000(userdata:grow)
```

Virtual A/B Layout:

```
0x00002000@0x00004000(uboot)
0x00002000@0x00006000(misc)
0x00020000@0x00008000(init_boot_a)
0x00020000@0x00028000(init_boot_b)
0x00400000@0x00048000(super)     -- contains dynamic partitions
0x00002000@0x00448000(vbmeta_a)
0x00002000@0x0044a000(vbmeta_b)
-@0x0044c000(userdata:grow)
```

Key Considerations: - RK3588 typically boots from eMMC or SD card; NVMe is secondary. - Rockchip U-Boot must be configured for A/B slot selection. - `misc` partition stores boot control information. - `super` partition uses dynamic partitions for Virtual A/B.

6.4 Rockchip (RK3588) Update Tools and Mechanisms

Tools: 1. **RKDevTool (Windows):** GUI-based firmware flashing tool for Rockchip devices. - Uses Rockchip USB download mode (Maskrom/Loader mode). - Flashes complete `update.img` to eMMC.

1. **upgrade_tool (Linux):** Command-line firmware flashing tool.
   - Linux equivalent of RKDevTool.
   - Can flash `update.img` or individual partitions.
2. **SD_Firmware_Tool:** Creates bootable SD cards for firmware update.
   - Device boots from SD card and flashes `update.img` to eMMC.
3. **rkflash**: Kernel module for accessing Rockchip eMMC from Linux.

Rockchip update.img Format: - Rockchip's custom firmware image format. - Contains all partitions in a single file. - Includes a firmware header with partition table. - Generated by `afptool` and `rkImageMaker` from Rockchip SDK.

OTA Mechanism (Rockchip Android): 1. Build `recovery.img` with Rockchip's custom recovery. 2. Generate OTA package using `ota_from_target_files`. 3. Push OTA zip to device (`/data/ota/` or `/cache/`). 4. Recovery mode applies the update. 5. Uses Rockchip's custom `updater` binary.

RK3588 OTA Build Steps (from Neardi community): 1. Check for userdata partition (required for storing OTA package). 2. Compile recovery-related programs: `make recoveryimage`. 3. Generate OTA package: `make otapackage` or `ota_from_target_files`. 4. Push OTA zip to device. 5. Apply via recovery: `adb reboot recovery` or `Settings → System Update`.

6.5 Recovery-Based vs A/B Update for RK3588

Recovery-Based (Current Default): - ✅ Simpler to implement. - ✅ Lower storage requirements. - ❌ Downtime during update. - ❌ Risk of bricking. - ❌ No automatic rollback. - ❌ Deprecated in Android 15.

A/B Update: - ✅ No downtime. - ✅ Automatic rollback. - ✅ Android 15 compatible. - ❌ Higher storage requirements (dual partitions). - ❌ Requires modified partition layout. - ❌ Requires A/B-capable bootloader.

Virtual A/B: - ✅ No downtime. - ✅ Automatic rollback. - ✅ Android 15 mandated. - ✅ Lower storage overhead than legacy A/B. - ❌ Most complex to implement. - ❌ Requires dm-snapshot kernel support. - ❌ Requires super partition with dynamic partitions. - ❌ Requires significant Rockchip BSP modification.

Recommendation for RK3588: - Start with **legacy A/B** (simpler to implement, Android 15 compatible). - Plan migration path to Virtual A/B. - Recovery-based is not viable for production.

6.6 Build Artifacts: OTA Zip Format, Flashing Images, Hash Verification

OTA Zip Format:

```
ota_update.zip
├── META-INF/
│   └── com/
│       └── android/
│           ├── metadata          # Build info, device type, pre-
device
│           ├── otacert           # OTA signing certificate
│           └── updater           # edify updater binary (non-A/B)
├── payload.bin                   # Update payload (A/B / Virtual
A/B)
├── payload_properties.txt        # Payload metadata (offset,
size, hash)
├── care_map.pb                   # dm-verity care map
└── apex/                         # APEX module updates (if any)
```

payload.bin Structure:

```
[Header]
  - Magic: "CrAU"
  - Major version: 2
  - Manifest size (varint)
  - Metadata signature size (varint)
[Manifest (protocol buffer)]
  - Partition operations list
  - For each partition:
    - Partition name, old/new partition info
    - List of operations (REPLACE, SOURCE_COPY, SOURCE_BSDIFF,
```

```
etc.)
    - Source/target digest (SHA-256)
[Metadata Signature]
  - RSA signature over manifest
[Extra Data]
  - Data blobs referenced by operations
```

payload_properties.txt:

```
FILE_HASH=<sha256_of_payload>
FILE_SIZE=<payload_size>
METADATA_HASH=<sha256_of_manifest>
METADATA_SIZE=<manifest_size>
```

care_map.pb: - Protocol buffer defining which blocks need dm-verity verification after update. - Used by update_engine to mark verity-protected regions.

Flashing Images: - `boot.img`: Kernel + ramdisk (or `init_boot.img` for Android 13+). - `system.img`: System partition (or part of `super.img`). - `vendor.img`: Vendor partition (or part of `super.img`). - `super.img`: Contains dynamic partitions (system, vendor, product, odm). - `vbmeta.img`: Verified Boot metadata. - `dtbo.img`: Device Tree Blob Overlay. - `update.img`: Rockchip all-in-one firmware image (contains all partitions).

Hash Verification: - OTA package: SHA-256 of payload.bin verified against payload_properties.txt. - Payload manifest: RSA signature verified against OTA certificate. - Individual partitions: Source and target SHA-256 digests in manifest. - dm-verity: Runtime integrity verification of system/vendor partitions. - AVB (Android Verified Boot): Boot-time verification chain.

---

7. Comparative Analysis

7.1 Feature Matrix

Feature	AOSP Update Engine	Mender	RAUC	SWUpdate	hawkBit	Ubuntu Core	Cl
A/B Updates	✅	✅	✅	✅	N/A (server)	✅	✅
Virtual A/B	✅	❌	❌	❌	N/A	❌	❌
Automatic Rollback	✅	✅	✅	✅	❌ (delegated)	✅	✅
Delta Updates	✅	⚠️ (Enterprise)	❌	❌	N/A	✅	✅
Phased Rollout	✅ (Omaha)	✅ (Enterprise)	❌ (via hawkBit)	❌ (via hawkBit)	✅	⚠️ (channels)	✅
Cryptographic Signing	✅ (RSA)	✅ (RSA)	✅ (X.509)	✅ (RSA/ CMS)	❌ (delegated)	✅ (assertions)	✅
	✅	✅	✅	✅	N/A	✅	✅

Feature	AOSP Update Engine	Mender	RAUC	SWUpdate	hawkBit	Ubuntu Core	Cl
Open Source Client							
Open Source Server	❌	✅	❌	❌	✅	❌	❌
Android Support	✅	❌	❌	❌	⚠️ (custom)	❌	❌
Fleet Management	❌ (server proprietary)	✅	❌ (via hawkBit)	❌ (via hawkBit)	✅	⚠️	❌
Device Inventory	❌	✅	❌	❌	✅	❌	❌

7.2 Licensing Comparison

Solution	Client License	Server License	Commercial Aspects
AOSP Update Engine	Apache 2.0	Proprietary (Google)	N/A
Mender	Apache 2.0 / MIT	Apache 2.0 (OSS) + Commercial (Enterprise)	Enterprise features paywalled
RAUC	LGPL-2.1	N/A (server-agnostic)	Commercial support from Pengutronix
SWUpdate	GPL-2.0+	N/A (server-agnostic)	N/A
hawkBit	N/A	EPL-2.0	N/A
Ubuntu Core	GPL-3.0	Proprietary (Canonical)	Snap Store is proprietary
AWS IoT	AWS SDK	Proprietary	Pay-per-use
Azure IoT	MIT (agent)	Proprietary	Azure pricing
Memfault	MIT/BSD	Proprietary	Commercial SaaS
JFrog Connect	Proprietary	Proprietary	Commercial

7.3 Encapsulability Assessment

Solution	Encapsulatable?	Approach
AOSP Update Engine	⚠️ Partial	Replace Omaha server; keep payload format and client
Mender	✅ Yes	Well-documented REST API; custom client possible

Solution	Encapsulatable?	Approach
RAUC	✅ Yes	Server-agnostic by design
SWUpdate	✅ Yes	Modular handler architecture
hawkBit	✅ Yes	Client-agnostic DDI API
Ubuntu Core	❌ No	Tightly coupled to snapd and Snap Store
AWS IoT	⚠️ Partial	Can use as transport; device logic is custom
Azure IoT	⚠️ Partial	Open-source agent (MIT); Azure-specific protocol

8. Recommendations for Custom OTA Design

8.1 Architecture Recommendations

Based on the research, the recommended approach for a custom OTA system targeting Android 15 on RK3588 (Orange Pi 5 Max):

1. **Use Android's update_engine as the update client.** It is the only mechanism that properly supports Virtual A/B (required for Android 15) and is already part of AOSP. Do not reinvent the client-side update mechanism.

2. **Build a custom OTA server** that speaks a simplified protocol (not Omaha XML). The server should:

   - Provide a REST API for update checks (JSON instead of XML).
   - Host OTA payloads on CDN-compatible storage.
   - Manage device inventory and deployment groups.
   - Support phased rollouts (percentage-based).
   - Track deployment status per device.

3. **Wrap/encapsulate the update_engine client** by:

   - Replacing the Omaha update check with a custom update check service.
   - Using `update_engine_client` CLI or DBus API to trigger updates.
   - Providing OTA payloads in the standard format (payload.bin + payload_properties.txt + care_map.pb).

4. **Implement A/B partition layout on RK3588** (start with legacy A/B, plan for Virtual A/B):

   - Modify `parameter.txt` for A/B partitions.
   - Configure Rockchip U-Boot for A/B slot selection.

    - Implement boot_control HAL.

5. **Use hawkBit as the server backend** (optional) for fleet management, or build a lightweight custom server:
    - hawkBit provides phased rollout, device inventory, and deployment management.
    - Custom adapter would translate hawkBit DDI API to Android update_engine commands.

8.2 Key Design Decisions

Decision	Recommendation	Rationale
Update Client	AOSP update_engine	Only client supporting Virtual A/B; required for Android 15
Partition Scheme	Legacy A/B (initial), Virtual A/B (future)	A/B is simpler; Virtual A/B is the standard
Server Protocol	REST/JSON (custom)	Simpler than Omaha; easier to implement and maintain
Payload Format	Standard Android OTA (payload.bin)	Required for compatibility with update_engine
Signing	RSA-4096 with OTA certificate	Standard Android approach; proven security
Delta Updates	Supported via payload format	update_engine supports SOURCE_BSDIFF/ PUFFDIFF
Phased Rollout	Server-side percentage control	Simple, effective, hawkBit-compatible
Device Auth	JWT or mTLS	Industry standard; secure
Artifact Storage	S3-compatible object storage	CDN-compatible, scalable
Fleet Management	hawkBit or custom	hawkBit is mature; custom is simpler

8.3 Critical Path for RK3588 Android 15 OTA

1. **Modify RK3588 partition layout** for A/B support (parameter.txt + U-Boot).
2. **Implement boot_control HAL** for RK3588 (may need custom implementation for Rockchip bootloader).
3. **Enable update_engine** in the Android 15 build for RK3588.
4. **Build OTA server** with REST API (update check, download URL, deployment management).

5. **Create update_engine integration layer** that bridges the custom server to update_engine.
6. **Implement device-side OTA app** (or modify System Updates UI) to show update status.
7. **Test full OTA flow:** Build → Sign → Upload → Deploy → Download → Apply → Reboot → Verify.
8. **Add rollback testing:** Force failed boots, verify automatic rollback.
9. **Implement phased rollout** in the OTA server.
10. **Add monitoring and reporting:** Device health, update success rates, failure analysis.

8.4 Open Questions / Risks

1. **Rockchip U-Boot A/B support:** Does the Rockchip U-Boot properly support A/B slot selection? May require custom patches.
2. **Virtual A/B on RK3588:** dm-snapshot and dm-user kernel module support in Rockchip's kernel. May need custom kernel configuration.
3. **super partition on RK3588:** Dynamic partitions require proper super partition configuration. Rockchip BSP may need modification.
4. **AVB on RK3588:** Android Verified Boot 2.0 support. Rockchip has AVB support but may need proper key provisioning.
5. **OTA package signing:** Need to generate and provision OTA signing keys for the device.
6. **Storage constraints:** A/B requires ~2x storage for system partitions. eMMC size on Orange Pi 5 Max may be a constraint.

Appendix A: Key References

- AOSP OTA Updates Documentation
- A/B (Seamless) System Updates
- Virtual A/B Overview
- OTA Tools (ota_from_target_files)
- ChromeOS Update Engine README
- Mender Architecture
- RAUC Documentation
- SWUpdate
- Eclipse hawkBit
- rpm-ostree
- Flatcar Update Strategies
- Azure IoT Device Update
- RK3588 OTA Upgrade Guide
- OTA Comparison: RAUC vs SWUpdate vs Mender
- Android Update Engine Deep Dive

Appendix B: Glossary

- **A/B Updates:** Update mechanism using two sets of partitions (slots), allowing seamless updates.
- **Virtual A/B:** Enhanced A/B using dm-snapshot/COW to reduce storage overhead.

- **Omaha Protocol:** Google's XML-based update check protocol.
- **update_engine:** Android/ChromeOS daemon for applying A/B updates.
- **payload.bin:** Binary update payload used by update_engine.
- **dm-verity:** Device-mapper verity target for runtime filesystem integrity verification.
- **AVB:** Android Verified Boot — boot-time integrity verification.
- **Dynamic Partitions:** Android partition scheme where system/vendor/product are logical partitions within a `super` partition.
- **COW:** Copy-on-Write — technique used by Virtual A/B to track changes during update.
- **OTA:** Over-The-Air — remote update delivery mechanism.
- **Delta Update:** Update that only includes changes from previous version (smaller download).
- **Care Map:** Protocol buffer describing which blocks need dm-verity verification after update.
- **edify:** Scripting language used in Android recovery-mode updater.
- **DDI API:** Device Direct Integration API — hawkBit's device-facing REST API.